Integración por Monte Carlo en memoria compartida con OpenMP

Introducción a la computación paralela · Universidad Industrial de Santander · 2026-2

Sebastian Sandoval · código 2211853

Repositorio: https://github.com/DaGuishe/ExamenOpenMP

La presentación de Tesis en 3 minutos va en un archivo aparte, dentro del mismo repositorio.

> **Resultados principales** con la semilla 2211853 y N = 10^8.
>
> $\pi \approx$ **3,141673**, con un error de 0,0000803. El error típico que se espera para ese N es 0,000164.
>
> Los aciertos fueron **78.541.825** con 1, 2, 4, 8 y 10 hilos, en las 63 corridas de la Parte III. El número de hilos no cambia el resultado.
>
> Speedup S(2) = 1,97, S(4) = 3,80 y **S(8) = 6,87**. La fracción paralelizable estimada es **f = 0,983**.
>
> Todo corrió en mi portátil, un MacBook Air con chip Apple M5 de 10 núcleos, 4 de alto rendimiento y 6 de eficiencia, con macOS 26.5, clang 21 y libomp 23. Las mediciones se hicieron con batería. El código y las salidas crudas quedan en el repositorio.

Parte I. Fundamento del método

Se lanzan puntos al azar sobre un cuadrado de lado 1, como quien tira dardos sin apuntar, y se cuenta cuántos caen dentro del cuarto de círculo de radio 1. El área de ese cuarto de círculo es $\pi/4$ y la del cuadrado es 1, entonces la fracción de aciertos se acerca a $\pi/4$ y

$$\pi \approx 4 \times \text{puntos dentro} \div \text{puntos totales}$$

Con los primeros 2.000 puntos de mi semilla cayeron 1.559 dentro, lo que da $\pi \approx 3{,}118$. Es una estimación pobre, y es normal con tan pocos puntos. Con 100 millones de puntos el valor sube a **$\pi \approx$ 3,141673**.

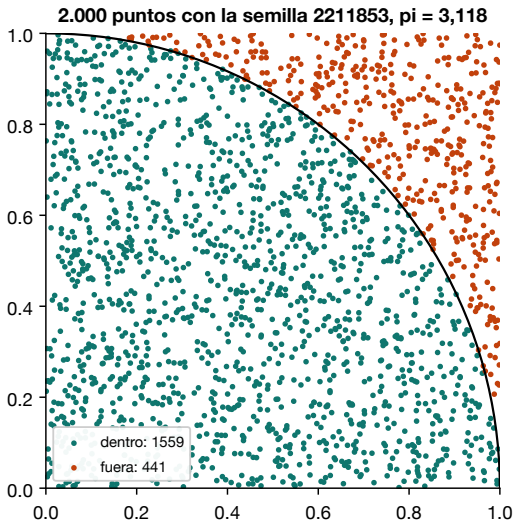


Figura 1. Los primeros 2.000 puntos con la semilla 2211853. En verde los que caen dentro del cuarto de círculo y en naranja los que quedan por fuera.

> **I.1.** Explique con sus propias palabras por qué el error del estimador disminuye en proporción a $1/\sqrt{N}$ y deduzca cuántas muestras adicionales se requieren para reducir el error a la mitad. ¿Qué implicación tiene esto sobre el costo de ganar un dígito de precisión?

Para entender el $1/\sqrt{N}$ sirve pensar que cada punto funciona como una moneda cargada. Cae dentro con probabilidad $q = \pi/4 \approx 0{,}785$ y por fuera con probabilidad $1 - q$. Un punto solo no dice casi nada. Al promediar muchos, la mala suerte de unos se compensa con la buena suerte de otros, pero esa compensación nunca es completa. La varianza del promedio de N ensayos independientes es $q(1 - q)/N$, y su raíz, que es la que mide el error, baja con $\sqrt{N}$ y no con N. Al multiplicar por 4 queda

$$\text{error típico} \approx 4\,\sqrt{(\,q(1-q)\,/\,N\,)} \approx 1{,}64\,/\,\sqrt{N}$$

Lo quise ver con mis propios números. Para cada N repetí el cálculo 30 veces. En cada repetición usé un tramo distinto y sin cruces de la misma secuencia de números aleatorios, saltando con la función que explico en la Parte II. Después promedié el error absoluto. Hay un detalle que me costó entender. El promedio del error absoluto no es igual a la desviación típica, sino unas 0,80 veces ella, porque para una normal la media del valor absoluto vale $\sqrt{(2/\pi)} \approx 0{,}80$ desviaciones. Mis datos quedaron entre 0,76 y 0,88 veces $1{,}64/\sqrt{N}$, justo alrededor de ese 0,80. Con $N = 10^6$ el error medio fue 0,00145 y la fórmula da 0,00164 para la desviación y 0,00131 para el error medio esperado.

Cuántas muestras más para la mitad del error. Si el error es $1{,}64/\sqrt{N}$, reducirlo a la mitad exige duplicar $\sqrt{N}$, y eso significa multiplicar N por 4. Partiendo de N muestras, toca agregar **3N muestras adicionales**.

Qué cuesta un dígito. Ganar un dígito es dividir el error por 10, y eso pide 100 veces más muestras. En mi equipo 10^8 puntos tardan 0,545 s con un hilo. Un dígito más serían unos 55 s y otro más, cerca de hora y media. Además, mi generador repite su ciclo pasados unos $7{,}2 \times 10^8$ puntos, y ni siquiera podría llegar ahí sin cambiar de generador. Con este método cada dígito adicional cuesta cien veces el tiempo del anterior.

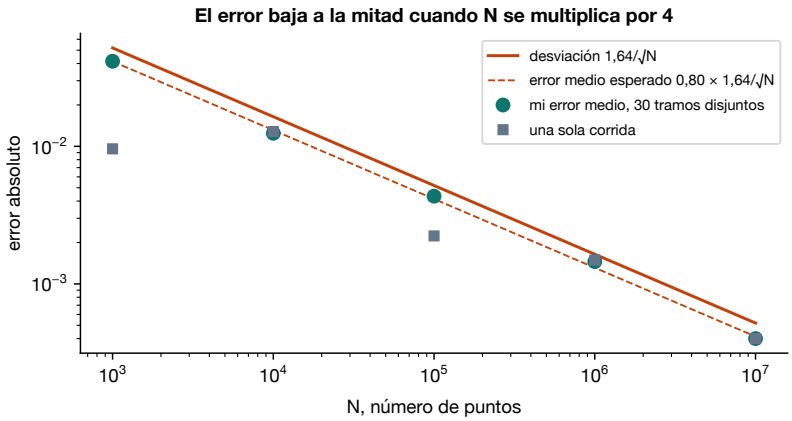


Figura 2. Error frente a N en escala logarítmica. Los puntos verdes son el promedio de 30 corridas en tramos disjuntos y caen sobre la línea punteada del error medio esperado. Los cuadros grises son una sola corrida, que puede tener más o menos suerte.

> **I.2.** Justifique por qué la integración por Monte Carlo se clasifica como un problema embarazosamente paralelo: analice las dependencias entre las muestras (o su ausencia) y determine qué datos deben ser compartidos y cuáles deben ser privados cuando el muestreo se reparte entre varios hilos de ejecución.

Se le dice embarazosamente paralelo porque repartirlo casi no cuesta trabajo. Ningún punto depende de otro. Para saber si el punto 7.000.000 cae dentro no necesito saber nada del 6.999.999. Los hilos no se esperan ni se pasan datos mientras calculan, y lo único que se junta es el total de aciertos, una vez y al final. La única dependencia escondida está en el generador de números aleatorios, porque cada número sale del anterior. Por eso el estado del generador no puede ser compartido.

Qué se comparte y qué es privado. Siguiendo la regla vista en clase, lo declarado antes de la región paralela es compartido y lo declarado dentro es privado. La Figura 3 lo resume.

Dato	Tipo	Razón
N, número de bloques, semilla base	Compartidos	Todos los hilos los leen y ninguno los modifica
Estado del generador	Privado	Si fuera compartido, los hilos se pisarían al pedir números
x, y, índices del ciclo	Privados	Cada hilo calcula sus propios puntos
Contador de aciertos	Privado y sumado al final	Con reduction(+:dentro) cada hilo cuenta en su copia y OpenMP las suma

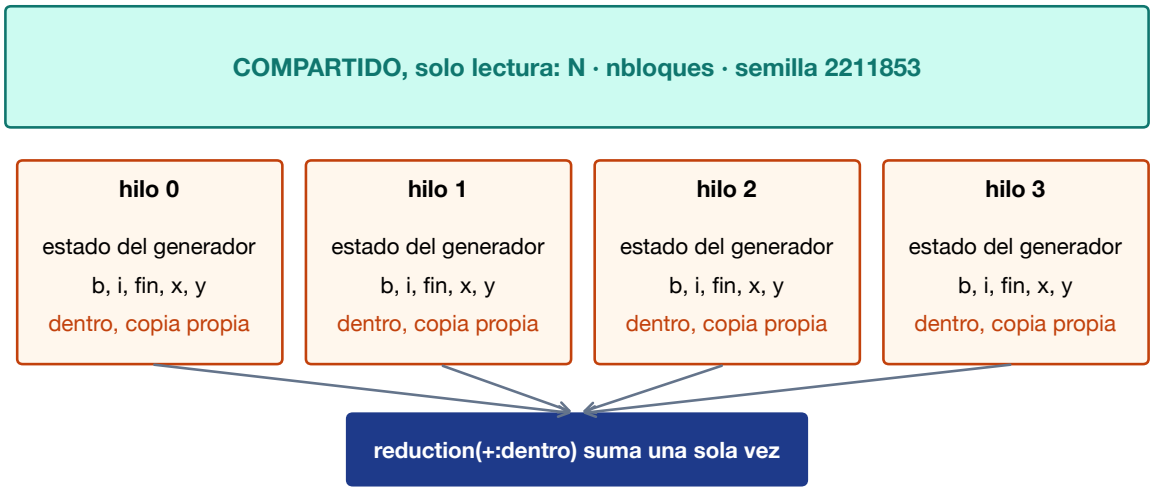


Figura 3. Arriba lo que ven todos los hilos, en el medio lo que tiene cada uno y abajo la suma final.

> **I.3.** Demuestre que repartir las N muestras entre p hilos y sumar los aciertos parciales de cada uno produce el mismo estimador que la versión secuencial. Con el mismo N total, ¿qué precisión estadística cabe esperar respecto de la ejecución con un solo hilo? Justifique.

Demostración. Sea a_i igual a 1 si el punto i cae dentro y 0 si no. La versión secuencial calcula dentro = $a_1 + a_2 + \ldots + a_N$. Si los N puntos se parten en p grupos disjuntos que entre todos cubren los N puntos, el hilo k obtiene un parcial d_k, que es la suma de los a_i de su grupo. Entonces

$$d_1 + d_2 + \ldots + d_p = a_1 + a_2 + \ldots + a_N = \text{dentro}$$

porque es la misma suma agrupada de otra forma, y la suma de enteros es asociativa y conmutativa. Como 4 · dentro / N usa el mismo numerador y el mismo N, el estimador es exactamente el mismo. Al ser un contador entero, ni siquiera aparecen las diferencias de redondeo que sí salen cuando se reducen números de punto flotante en otro orden. La condición importante es que cada punto i tenga siempre los mismos x e y, sin importar qué hilo lo procese. Mi programa lo garantiza atando los números aleatorios a la posición del punto y no al hilo.

Lo comprobé partiendo los 10^8 puntos en 4 tramos de 25 millones: 19.637.752 + 19.633.631 + 19.633.530 + 19.636.912 = **78.541.825**, el mismo conteo de la versión secuencial.

Precisión con p hilos. Es la misma que con uno. La precisión depende de cuántos puntos distintos se usan, no de cuántos hilos los procesan. Con $N = 10^8$ el error típico es 0,000164 con 1 hilo o con 10. Lo único que cambia es el tiempo. Esto se cumple solo si los hilos usan puntos distintos. Si todos arrancaran con la misma semilla habría apenas N/p puntos diferentes y el error crecería en un factor $\sqrt{p}$, como muestro en II.3.

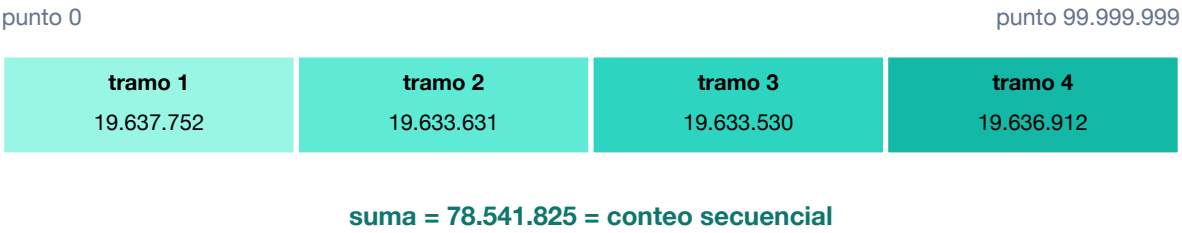


Figura 4. Los 10^8 puntos en 4 tramos contiguos. La suma de los parciales coincide con el conteo secuencial.

```
$ ./tramos
tramo 1: puntos 0 a 24999999, estado inicial 2211853, aciertos 19637752
tramo 2: puntos 25000000 a 49999999, estado inicial 2913319565, aciertos 19633631
tramo 3: puntos 50000000 a 74999999, estado inicial 3863000333, aciertos 19633530
tramo 4: puntos 75000000 a 99999999, estado inicial 66236301, aciertos 19636912
suma de los 4 tramos = 78541825
secuencial           = 78541825
```

Figura 5. Ejecución real en mi computador del reparto en 4 tramos. El estado inicial de cada tramo se calcula con el salto del generador.

Parte II. Implementación en OpenMP

El enunciado trae la semilla 20261018, pero el profesor indicó que cada estudiante usa su código como semilla. Por eso todos mis programas usan **2211853**.

Hubo un problema antes de empezar. Mi computador es un Mac, y el rand_r de la biblioteca de macOS no usa el mismo algoritmo que el de glibc en Linux. Con la misma semilla daría otra secuencia y mis resultados no se podrían comparar con los de nadie del curso. Para arreglarlo escribí `rand_r_glibc`, una copia exacta del algoritmo de glibc, que hace tres pasos del generador congruencial $s \leftarrow 1103515245 \cdot s + 12345$ por llamada, combinando 11, 10 y 10 bits. Para validarla usé un número conocido. Con la semilla 2224510 y $N = 10^8$, una compañera obtuvo en Linux 78.541.707 aciertos, y mi función, corriendo en el Mac, dio exactamente 78.541.707. Con eso quedé seguro de que el generador es idéntico bit a bit.

> **II.1.** Compile y ejecute la versión secuencial con $N = 10^8$. Verifique que la estimación es consistente con el valor de π (un error del orden de 10^{-4} es normal; explique de dónde sale ese orden de magnitud). Registre el tiempo de ejecución T(1) con omp_get_wtime() y consérvelo para la parte III.

Tomé el programa del enunciado, cambié la semilla por 2211853 y la llamada a rand_r por rand_r_glibc, y le agregué `omp_get_wtime()` alrededor del ciclo. El resto quedó igual. Imprimió $\pi \approx 3{,}141673$, con 78.541.825 de los 10^8 puntos dentro. Como el valor real es 3,141593, el error queda en **0,0000803**.

De dónde sale el orden de 10^{-4}. Por la Parte I, el error típico es $1{,}64/\sqrt{N}$. Con $N = 10^8$, $\sqrt{N} = 10.000$ y el error esperado es cerca de 0,000164, que es del orden de 10^{-4}. Mi error queda por debajo de ese valor y por eso la estimación es consistente.

Tiempo. Corrí el secuencial 7 veces dentro del protocolo de la Parte III, con una corrida de calentamiento antes de cada ronda. Los tiempos fueron de 0,5445 s a 0,5464 s y la mediana es **T(1) = 0,5451 s**. Ese es el valor que uso en la Parte III.

```
$ clang -O2 -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include secuencial.c -o secuencial -
L$(brew --prefix libomp)/lib -lomp -lm
$ ./secuencial 100000000
secuencial N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.5449 s
```

Figura 6. II.1 en mi equipo. El programa del enunciado con mi semilla da $\pi = 3{,}141673$ y tarda unos 0,545 s.

> **II.2.** Implemente la versión paralela con OpenMP. Su programa debe:
>
> - usar una directiva omp parallel for con la cláusula de reducción adecuada sobre el acumulador de aciertos;
> - permitir elegir el número de hilos p (con omp_set_num_threads o OMP_NUM_THREADS);
> - garantizar que cada hilo disponga de un flujo de números aleatorios independiente y sin solapamiento con el de los demás. Como semilla base use la que su docente le asignó (las semillas se publican por estudiante) y espacie las de cada hilo al menos N/p valores. El resultado debe ser reproducible: con las mismas semillas, el programa debe producir la misma estimación cualquiera que sea el valor de p.

Dejé el ciclo del profesor tal cual, con la misma prueba $x^2 + y^2 \le 1$, dos llamadas al generador por punto y la misma división por RAND_MAX. Lo nuevo está alrededor. Los 10^8 puntos se parten en **400 bloques de 250.000** y OpenMP reparte los bloques entre los hilos.

```
#pragma omp parallel for reduction(+:dentro) schedule(runtime)
for (long b = 0; b < nbloques; b++) {
    unsigned int estado =
        avanzar_lcg(semilla, (unsigned long long)b * TAM_BLOQUE * PASOS_MUESTRA);
    /* ciclo del programa del profesor, desde b*TAM_BLOQUE hasta fin */
}
```

- **Reducción.** `reduction(+:dentro)`. Cada hilo cuenta en una copia privada que arranca en cero y OpenMP suma las copias al cerrar el ciclo.
- **Número de hilos.** Va como segundo argumento, por ejemplo `./montecarlo 100000000 4`, y el programa llama a `omp_set_num_threads`. Si no se pasa, se respeta `OMP_NUM_THREADS`.
- **Flujos sin solapamiento.** Todos los números salen de una única secuencia que empieza en 2211853. Cada punto consume 6 pasos del generador. Entonces el bloque b empieza $6 \cdot 250.000 \cdot b$ pasos después de la semilla. La función `avanzar_lcg` calcula ese estado sin recorrer los pasos uno por uno. Aprovecha que aplicar k veces $s \leftarrow a{\cdot}s + c$ sigue siendo una recta $A{\cdot}s + C$, y arma A y C por cuadrados sucesivos en unas 30 iteraciones. Lo pensé como un libro largo del que cada bloque lee su propio capítulo, que nunca se cruza con el de otro.
- **Espaciado de al menos N/p.** Con `schedule(static)`, que es el que uso por defecto y en todas las mediciones, cada hilo recibe bloques contiguos. Con 4 hilos el hilo k empieza en el bloque 100k, es decir, 25.000.000 de puntos después del anterior, que es justo N/p en puntos. Medido en valores del generador son 150 millones de pasos, seis veces el mínimo que pide el enunciado. Con `dynamic` los

tramos tampoco se cruzan, pero dos hilos pueden quedar separados por un solo bloque. Por eso la letra del requisito solo la cumplo con static.

- **Mismo resultado con cualquier p.** Los números dependen del bloque y no del hilo. Por eso el conteo fue **78.541.825** con 1, 2, 4, 8 y 10 hilos, con static y con dynamic, y con `OMP_NUM_THREADS`. Coincide con el secuencial. Con 8 hilos el tiempo bajó de 0,55 s a 0,08 s.

Elegí 400 bloques porque se dividen exacto entre 1, 2, 4, 8 y 10 hilos y ningún hilo queda con trabajo de más. El programa también revisa al arrancar que saltar 6 pasos dé lo mismo que dos llamadas seguidas al generador, y se niega a correr si N supera los 715.827.882 puntos, porque ahí la secuencia de 2^{32} valores empezaría a repetirse.

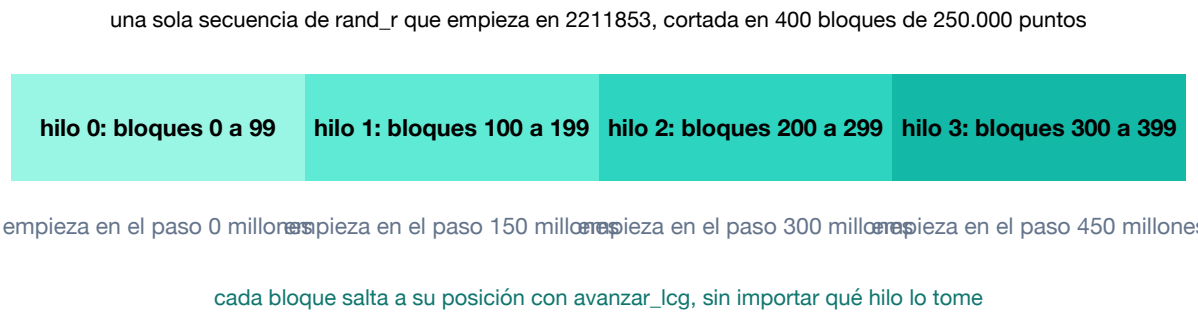


Figura 7. Reparto con 4 hilos. Cada hilo toma 100 bloques seguidos de la misma secuencia.

```
$ clang -O2 -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include montecarlo.c -o montecarlo -
L$(brew --prefix libomp)/lib -lomp -lm
$ ./montecarlo 100000000 1
hilos=1 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.5495 s
$ ./montecarlo 100000000 2
hilos=2 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.2760 s
$ ./montecarlo 100000000 4
hilos=4 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.1429 s
$ ./montecarlo 100000000 8
hilos=8 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.0778 s
$ ./montecarlo 100000000 10
hilos=10 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.0692 s
$ OMP_NUM_THREADS=8 ./montecarlo
hilos=8 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.0794 s
$ gcc-16 -O2 -fopenmp montecarlo.c -o montecarlo_gcc -lm
$ ./montecarlo_gcc 100000000 8
hilos=8 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.1296 s
```

Figura 8. II.2 con 1, 2, 4, 8 y 10 hilos, por argumento y con OMP_NUM_THREADS. Al final, el comando literal del enunciado con gcc 16 compila sin advertencias y da el mismo conteo.

Con gcc el programa corre cerca de un 35 % más lento que con clang en este equipo, aunque el resultado es idéntico. En tres corridas después de calentar, gcc tardó 0,762 s con un hilo frente a 0,568 s de clang, y 0,108 s frente a 0,079 s con 8 hilos. Para las mediciones usé la versión de clang. El comando del enunciado lo dejé verificado para que el programa funcione igual en Linux.

> **II.3.** Explique por qué no es correcto que todos los hilos llamen a la función rand() compartiendo el estado global del generador. Su respuesta debe cubrir al menos dos problemas distintos: uno de condición de carrera sobre el estado interno, y otro de correlación entre secuencias si cada hilo inicializara el generador con la misma semilla. Indique qué consecuencias tendría cada uno sobre la estimación de π y sobre el error reportado.

rand() guarda su posición en una sola variable escondida que comparten todos los hilos. Para no quedarme en la teoría escribí `defectos.c`, que comete cada error a propósito con $N = 10^8$.

Problema 1. Condición de carrera sobre el estado. Es como si ocho personas escribieran al mismo tiempo en un solo cuaderno. Dos hilos pueden leer el mismo estado y sacar el mismo número, o uno puede borrar el avance del otro. Lo probé de dos maneras.

- **Estado compartido sin protección.** Con un hilo da los 78.541.825 de siempre. Con 8 hilos dio 78.537.419, 78.544.358 y 78.539.113 en tres corridas seguidas. Cambia cada vez, y en una corrida el error llegó a 0,000182, por encima del típico.
- **Estado compartido con candado.** Esto es lo que hace la biblioteca para que rand() no se dañe. El conteo queda correcto, pero los hilos hacen fila. Con un hilo la mediana fue 1,19 s y con 8 hilos **33,4 s**, unas 28 veces más lento.

Sobre π. Sin candado deja de ser reproducible y puede quedar sesgado, porque se repiten o se saltan números. Con candado el paralelismo desaparece.
Sobre el error reportado. La fórmula $1{,}64/\sqrt{N}$ supone 10^8 puntos independientes. Si hay números repetidos, en realidad hay menos puntos distintos y el error verdadero es mayor que el que se reporta.

Problema 2. Correlación por la misma semilla. Si cada hilo tiene su propio generador pero todos arrancan en 2211853, todos producen los mismos puntos. El resultado se volvió un múltiplo exacto de una corrida más corta.

- Con 4 hilos salieron 78.551.008 aciertos, que es exactamente $4 \times 19.637.752$, el conteo secuencial con N/4 puntos. $\pi = 3{,}142040$ y el error fue 0,000448.
- Con 8 hilos salieron $78.542.704 = 8 \times 9.817.838$, igual al secuencial con 12,5 millones de puntos.

Sobre π. El resultado depende de p, y en realidad solo hay N/p puntos distintos.
Sobre el error reportado. El error real crece como $\sqrt{p}$. Con 4 hilos se reportaría $\pm 0{,}000164$, cuando la precisión verdadera es la de 25 millones de puntos, $\pm 0{,}000328$. El 0,000448 que obtuve es del orden de ese $\pm 0{,}000328$ y casi el triple del 0,000164 que se reportaría.

Mi versión evita ambos problemas. Cada bloque tiene su propio estado privado y cada estado empieza en una posición diferente de la secuencia.

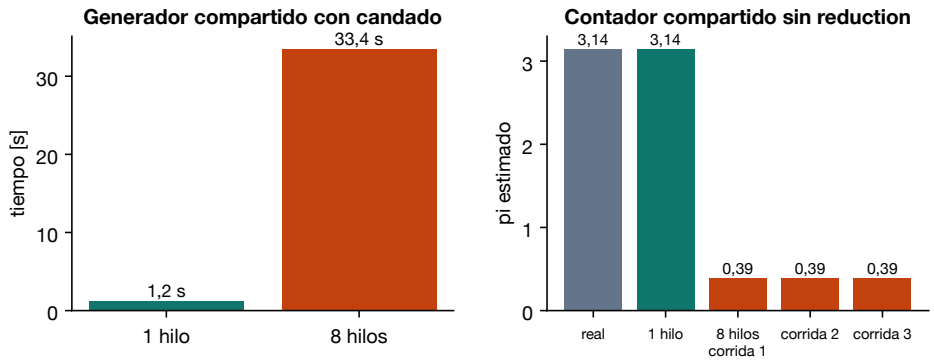


Figura 9. A la izquierda, con el generador compartido y protegido, 8 hilos tardan mucho más que uno. A la derecha, sin reduction, π queda muy por debajo del real, como explico en II.4.

> **II.4.** Analice los atributos de compartición de datos de su región paralela: enumere las variables que OpenMP trata como compartidas y como privadas por defecto en su código, explique la cláusula que usó para el acumulador y describa qué ocurriría si el acumulador se actualizara dentro del ciclo sin esa cláusula (diagnostique el tipo de defecto y su efecto sobre el resultado).

Para no adivinar copié la región en `default_none.c` y la compilé con `default(none)`, que obliga a declarar todo. El compilador marcó un error por cada variable compartida: `semilla`, `nbloques` y `N`, que es lo que muestra la Figura 11.

Variable	Atributo por defecto	Razón
N, nbloques, semilla	Compartidas	Se declaran antes del ciclo y solo se leen
b	Privada	Es la variable del ciclo del omp parallel for
estado, fin, i, x, y	Privadas	Se declaran dentro del cuerpo del ciclo
dentro	Compartida por defecto, privada por reduction	Es el contador de aciertos

Cláusula usada. `reduction(+:dentro)`. Cada hilo recibe una copia que empieza en 0, cuenta ahí sus aciertos y al final OpenMP suma las copias en la variable original. Sale mucho más barato que proteger cada acierto con critical o atomic, porque los hilos se coordinan una sola vez y no 78 millones de veces.

Sin la cláusula. `dentro++` parece una sola operación, pero por dentro el hilo lee el valor, le suma uno y lo guarda. Si dos hilos leen el mismo valor, los dos guardan el mismo resultado y un acierto se pierde, como en la Figura 10. Es una **condición de carrera** y el programa queda con comportamiento indefinido. En mi equipo el efecto fue peor que perder aciertos sueltos. Con un hilo dio bien, y por eso el defecto no se nota si uno solo prueba en secuencial. Con 8 hilos π quedó entre **0,3926 y 0,3929** en seis corridas. El conteo final, unos 9,82 millones, es casi exactamente la octava parte del real. Lo más probable es que clang haya guardado el contador en un registro de cada hilo y que al final cada uno escribiera su total encima del de los demás, de modo que solo sobrevivió el de un hilo. El resultado sale siempre por debajo del real y cambia un poco en cada ejecución.

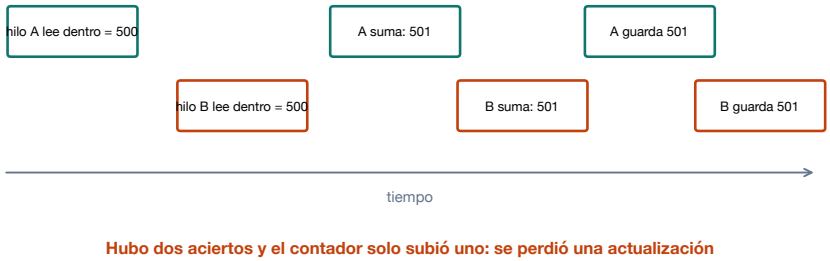


Figura 10. Cómo se pierde un acierto cuando dos hilos actualizan el mismo contador a la vez.

```
$ clang -fopenmp -fsyntax-only default_none.c
default_none.c:9:31: error: variable 'semilla' must have explicitly specified data sharing
attributes
default_none.c:8:26: error: variable 'nbloques' must have explicitly specified data sharing
attributes
default_none.c:10:40: error: variable 'N' must have explicitly specified data sharing attributes
$ ./defectos 4 8
modo=4 hilos=8 N=100000000 dentro=9820954 pi=0.392838 error=2.75e+00 tiempo=0.1127 s
$ ./defectos 4 8
modo=4 hilos=8 N=100000000 dentro=9820551 pi=0.392822 error=2.75e+00 tiempo=0.1126 s
$ ./defectos 4 8
modo=4 hilos=8 N=100000000 dentro=9821328 pi=0.392853 error=2.75e+00 tiempo=0.1120 s
$ ./defectos 4 1
modo=4 hilos=1 N=100000000 dentro=78541825 pi=3.141673 error=8.03e-05 tiempo=0.7547 s
```

Figura 11. II.4. default(none) nombra las variables compartidas. Sin reduction, con 8 hilos π sale cerca de 0,39 y con un hilo sale bien.

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

El speedup con p hilos es $S(p) = T(1)/T(p)$ y la eficiencia es $E(p) = S(p)/p$. Si una fracción f del trabajo es paralelizable, la ley de Amdahl da el speedup máximo

$$S(p) = 1 / ((1 - f) + f/p)$$

> **Protocolo experimental.** Con $N = 10^8$ fijo, mida $T(p)$ para $p \in \{1, 2, 4, 8\}$. Para cada valor de p realice al menos cinco ejecuciones y registre la mediana de los tiempos; justifique por qué la mediana es más apropiada que el promedio en este experimento.

Equipo. MacBook Air con Apple M5, que tiene **10 núcleos físicos sin hiperprocesamiento**. Cuatro son de alto rendimiento, que Apple llama Super, con 128 KiB de caché L1 de datos y 16 MiB de L2 compartida. Los otros seis son de eficiencia, con 64 KiB de L1 y 6 MiB de L2. Hay 16 GB de memoria. Como en macOS no existe lscpu, saqué los mismos datos con `sysctl` y `system_profiler`, que aparecen en la Figura 13. La salida completa está en `resultados/equipo.txt`. Medí con batería, sin modo de bajo consumo y con las demás aplicaciones cerradas.

Cómo medí. Hice 7 rondas. Cada ronda arranca con una corrida de calentamiento que descarto y luego mide el secuencial y el paralelo con 1, 2, 4, 8 y 10 hilos con static, más 4, 8 y 10 hilos con dynamic. El

orden de los casos rota de una ronda a otra para que ninguno quede siempre de primero. T(1) es el programa secuencial. Agregué p = 10 porque mi equipo tiene 10 núcleos y quería ver qué pasa cuando se usan todos.

p, hilos	Mediana de T(p) [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p
1	0,5451	1,000	1,000
2	0,2762	1,974	0,987
4	0,1433	3,804	0,951
8	0,0794	6,865	0,858
10	0,0674	8,088	0,809

Tabla 1. Mediana de 7 ejecuciones con $N = 10^8$ y schedule(static). Las 63 ejecuciones dieron 78.541.825 aciertos. El paralelo con 1 hilo tuvo mediana 0,5448 s, prácticamente igual al secuencial.

Por qué la mediana. El sistema operativo a veces interrumpe una corrida para atender otra cosa y la vuelve más lenta, nunca más rápida. Esas corridas raras arrastran el promedio, mientras que la mediana se queda con el valor del medio y casi no se mueve. En mis datos pasó con 8 hilos y dynamic. Seis corridas tardaron entre 78,2 y 78,4 ms y una tardó 106,6 ms. El promedio quedó en 82,4 ms y la mediana en 78,4 ms. Si hubiera usado el promedio, dynamic habría parecido más lento que static, y es al revés.

Hubo un caso más grave. En mi primera medición no calentaba el procesador y el secuencial siempre iba primero en cada ronda, justo después de una pausa. Le tocaba el núcleo todavía frío y daba 0,5717 s en lugar de 0,545. Eso infló T(1) y produjo S(2) = 2,07, más que lineal, y una f de 0,999. Un speedup mayor que p con un programa así no tiene sentido, y eso fue lo que me hizo sospechar. Repetí todo con calentamiento y orden rotativo, como muestra la Figura 12. Esa primera serie queda en el repositorio como evidencia, pero no la uso en los cálculos.

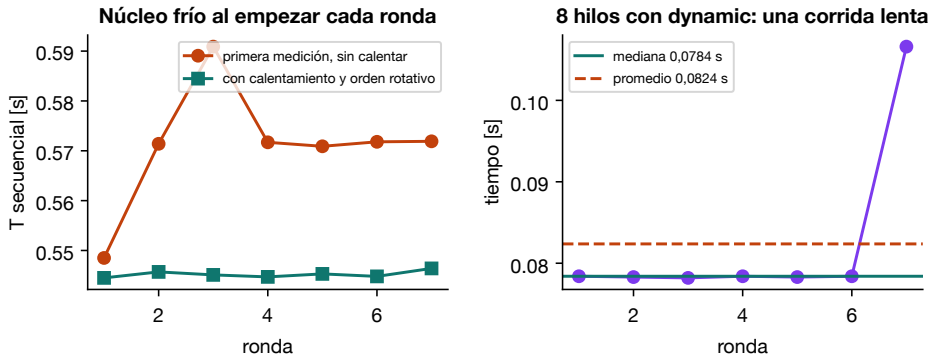


Figura 12. Izquierda: el tiempo secuencial en la primera medición, en naranja, y en la definitiva, en verde. Derecha: una corrida lenta de 8 hilos con dynamic sube el promedio pero casi no mueve la mediana.

```
$ system_profiler SPHardwareDataType | grep -E "Model Name|Chip|Cores|Memory" ; sysctl hw.ncpu
hw.physicalcpu hw.logicalcpu hw.perflevel0 hw.perflevel1 ...
Model Name: MacBook Air
Chip: Apple M5
Total Number of Cores: 10 (4 Super and 6 Efficiency)
Memory: 16 GB
hw.ncpu: 10
hw.physicalcpu: 10
hw.logicalcpu: 10
hw.nperflevels: 2
hw.perflevel0.name: Super
hw.perflevel0.physicalcpu: 4
hw.perflevel0.logicalcpu: 4
hw.perflevel0.l1dcachesize: 131072
hw.perflevel0.l2cachesize: 16777216
hw.perflevel1.name: Efficiency
hw.perflevel1.physicalcpu: 6
hw.perflevel1.logicalcpu: 6
hw.perflevel1.l1dcachesize: 65536
hw.perflevel1.l2cachesize: 6291456
hw.cachelinesize: 128
hw.memsize: 17179869184
machdep.cpu.brand_string: Apple M5
```

Figura 13. Datos del procesador. Hay 10 núcleos físicos y 10 lógicos, sin hiperprocesamiento, y dos tipos de núcleo.

```
$ python3 resumen.py
caso                      mediana      min      max    S(p)    E(p)
secuencial                 0.5451   0.5445   0.5464   1.000   1.000
paralelo p=1 static        0.5448   0.5446   0.5458   1.001   1.001
paralelo p=2 static        0.2762   0.2754   0.2766   1.974   0.987
paralelo p=4 static        0.1433   0.1428   0.1454   3.804   0.951
paralelo p=8 static        0.0794   0.0783   0.0804   6.865   0.858
paralelo p=10 static       0.0674   0.0658   0.0691   8.088   0.809
paralelo p=4 dynamic       0.1430   0.1429   0.1470
paralelo p=8 dynamic       0.0784   0.0782   0.1066
paralelo p=10 dynamic      0.0657   0.0651   0.0793
conteos distintos en 63 corridas: {'78541825'}
f = 0.983   Karp-Flatt e(2)=0.013 e(4)=0.017 e(8)=0.024 e(10)=0.026
```

Figura 14. Resumen de las 7 rondas: medianas, mínimos, máximos, S(p), E(p), f y Karp–Flatt.

> **III.1.** Aplique la ley de Amdahl a los siguientes casos y interprete cada resultado: (a) f = 0,98, p = 8; (b) f = 0,90, p = 8; (c) f = 0,98 y p creciendo sin límite. ¿Qué le enseña el caso (c) sobre la fracción secuencial de un programa?

Caso	Cálculo	Speedup	Lectura
a. f = 0,98, p = 8	1 / (0,02 + 0,1225) = 1 / 0,1425	**7,02**	Con 2 % secuencial, 8 núcleos rinden como 7. E = 0,88
b. f = 0,90, p = 8	1 / (0,10 + 0,1125) = 1 / 0,2125	**4,71**	Con 10 % secuencial se pierde más de un 40 %. E = 0,59
c. f = 0,98, p → ∞	1 / (0,02 + 0) = 1 / 0,02	**50**	Ni con infinitos núcleos pasa de 50

Lo que enseña el caso c. La parte secuencial pone un techo de $1/(1 - f)$ que no se rompe con más núcleos, como se ve en la Figura 15. Con un 2 % secuencial el techo es 50 y con un 10 % baja a 10. Pasar de 98 % a 90 % paralelizable parece un cambio menor, pero divide el techo por cinco. Si uno quiere acelerar un programa, rinde más reducir la parte secuencial que comprar núcleos.

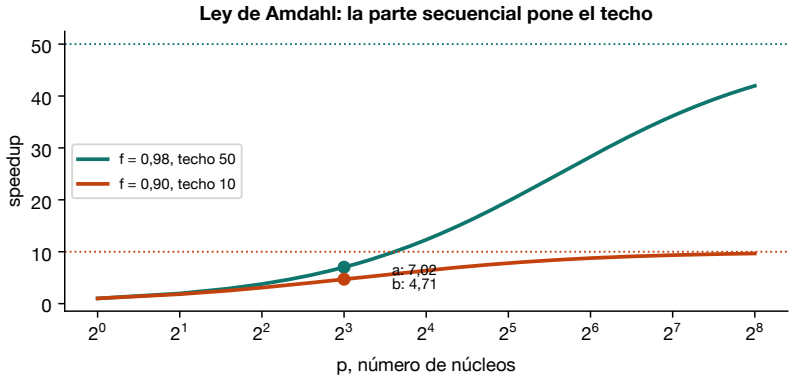


Figura 15. Ley de Amdahl para f = 0,98 y f = 0,90. Las curvas se aplanan contra su techo.

> **III.2.** Con sus mediciones, estime la fracción secuencial f de su programa: despeje f de la ley de Amdahl a partir del speedup medido con p = 4 y muestre el procedimiento completo. Si lo desea, contraste su resultado con la métrica de Karp–Flatt, $e = (1/S(p) - 1/p) / (1 - 1/p)$, y explique qué mide.

Con S(4) = 3,804, partiendo de Amdahl con p = 4:

$$1/S = (1 - f) + f/4 = 1 - 0{,}75\,f \;\Rightarrow\; f = (1 - 1/S) / 0{,}75 = (1 - 0{,}2629) / 0{,}75 = 0{,}7371 / 0{,}75 = \mathbf{0{,}983}$$

El 98,3 % del programa se comporta como paralelizable y la fracción secuencial aparente es **1 − f = 0,017**, un 1,7 %.

Karp-Flatt. Esta métrica calcula la fracción secuencial que haría falta para explicar el speedup que de verdad se midió con cada p. Me dio e(2) = 0,013, e(4) = 0,017, e(8) = 0,024 y e(10) = 0,026. Con p = 4 es igual a 1 − f, porque es la misma cuenta despejada de otra forma. Si hubiera un pedazo fijo de código imposible de repartir, e sería más o menos constante. En mi caso crece con p, y eso indica que el freno no es código secuencial sino costos que aumentan con el número de hilos. Lo secuencial de verdad en mi programa es mínimo. Crear los 4 hilos cuesta unos 33 μs frente a 143 ms de cálculo, un 0,02 %.

> **III.3.** Compare su speedup medido con el ideal lineal S(p) = p y con el predicho por la ley de Amdahl usando su f estimado. Explique las discrepancias que observe. Considere en su análisis, entre otros factores, la creación y destrucción de hilos, el costo de la reducción, los límites de ancho de banda de memoria y la jerarquía de caché, y el efecto de la frecuencia variable del procesador y del hiperprocesamiento.

p	Ideal	Amdahl, f = 0,983	Medido
2	2	1,97	1,97
4	4	3,80	3,80
8	8	7,14	6,87
10	10	8,66	8,09

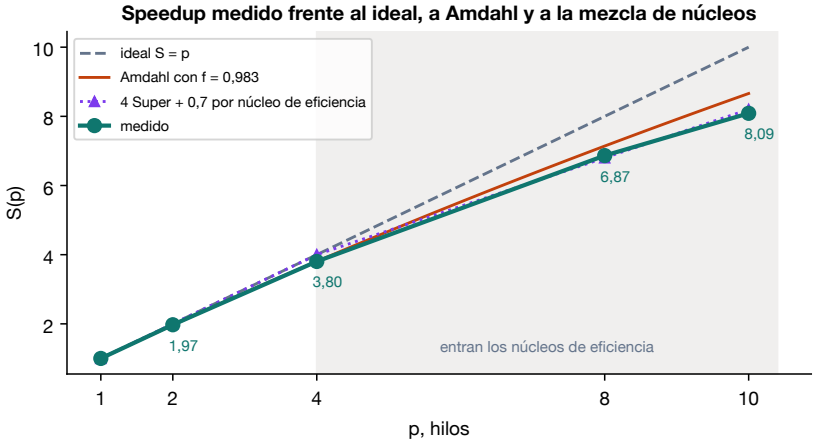


Figura 16. Speedup medido frente al ideal, a Amdahl y a un modelo que tiene en cuenta los dos tipos de núcleo. La zona sombreada es donde ya no alcanzan los núcleos Super.

Con 4 hilos Amdahl y lo medido coinciden en 3,80, pero eso no es un hallazgo, porque f se despejó justamente de ese dato. Hasta 4 hilos el programa casi va pegado al ideal, con eficiencia de 0,95. De ahí en adelante se despega del ideal y también de Amdahl. Con 8 hilos se esperaban 7,14 según Amdahl y medí 6,87. Revisé cada factor con una medición.

- **Crear y destruir hilos.** La primera región paralela, que es la que crea los hilos, costó 23 μs con 2 hilos, 33 μs con 4, 72 μs con 8 y 109 μs con 10, como muestra la Figura 17. Frente a 79 ms de cálculo con 8 hilos es menos del 0,1 %. No explica la diferencia.
- **Reducción.** Cada hilo suma una sola vez. Con 2 y 4 hilos el costo extra no se distingue del ruido. Con 8 y 10 sube a unos 19 y 26 μs, todavía despreciable.
- **Memoria y caché.** El programa no tiene arreglos. Cada hilo maneja el estado, x, y, el índice y su contador, que caben de sobra en la L1 de cualquiera de los dos tipos de núcleo. No hay tráfico a memoria que pueda saturar el ancho de banda, y como cada hilo escribe en su copia privada tampoco hay falso compartimiento.
- **Hiperprocesamiento.** El M5 no tiene, cada hilo de hardware es un núcleo completo. Lo que sí tiene mi equipo es una mezcla de núcleos rápidos y lentos, y esa resultó ser la causa principal.
- **Núcleos de distinto tipo.** Con 4 hilos todo indica que macOS los ubica en los 4 núcleos Super, porque la eficiencia es de 0,95 y cada hilo rinde como un núcleo rápido. Desde el quinto hilo entran los de eficiencia. Con `taskpolicy -c background` obligué al programa a correr solo en núcleos de eficiencia y un hilo tardó 1,930 s, contra 0,543 s en un núcleo Super, como se ve en la Figura 18. Ese modo además les baja la frecuencia, de modo que ese 3,6× es un caso extremo. Del propio speedup sale una cifra más realista. Pasar de 4 a 10 hilos sumó 4,28 de speedup con 6 núcleos de eficiencia,

unos 0,71 núcleos Super por cada uno. Ese 0,7 lo calibré con $p = 10$, así que ese punto no sirve para comprobar nada. El que sí sirve es $p = 8$, donde el modelo $4 + 0{,}7 \cdot (p - 4)$ predice 6,8 y medí 6,87. Amdahl supone núcleos iguales y por eso se queda corto.

- **Frecuencia variable.** En macOS la frecuencia solo se puede leer con `powermetrics`, que exige permisos de administrador, y por eso no la pude medir directamente. Sí encontré su efecto en la primera medición, la del núcleo frío, que salía un 5 % más lenta. Además el equipo es un MacBook Air sin ventilador y estaba con batería, de modo que con todos los núcleos ocupados por más tiempo podría bajar la frecuencia. En mis corridas, de menos de un segundo, no lo noté.

Y con 4 hilos, por qué 3,80 y no 4. Crear los hilos cuesta un 0,02 % y la reducción nada. Probé dynamic por si algún hilo terminaba antes que los otros y no cambió nada, 143,0 ms contra 143,3 ms, como muestra la Figura 19. Lo que queda, cerca de un 5 %, lo atribuyo a que macOS no me deja fijar los hilos a núcleos, porque libomp ignora `OMP_PROC_BIND` en este sistema. Lo comprobé con `OMP_DISPLAY_AFFINITY`, que reporta cada hilo como ligado a un conjunto de procesadores indefinido, según `resultados/afinidad.txt`. También influye que los procesos del sistema siguen ocupando ratos de los núcleos Super. Con 8 y 10 hilos dynamic sí ayudó un poco, 1,3 % y 2,5 %, porque los bloques pendientes se los llevan los núcleos que van más rápido.

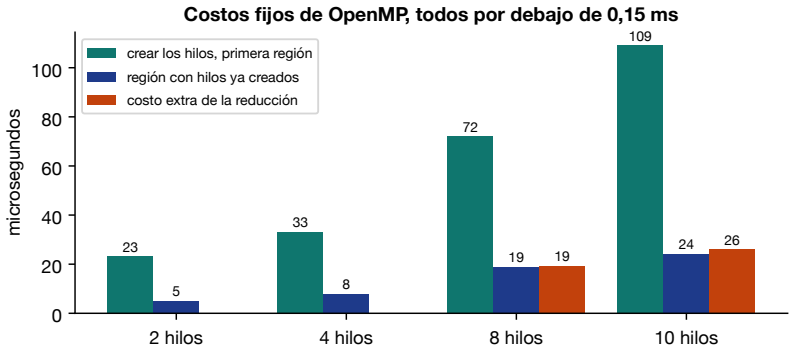


Figura 17. Costo de crear los hilos, de abrir una región con los hilos ya creados y de la reducción. Un μs es una millonésima de segundo.

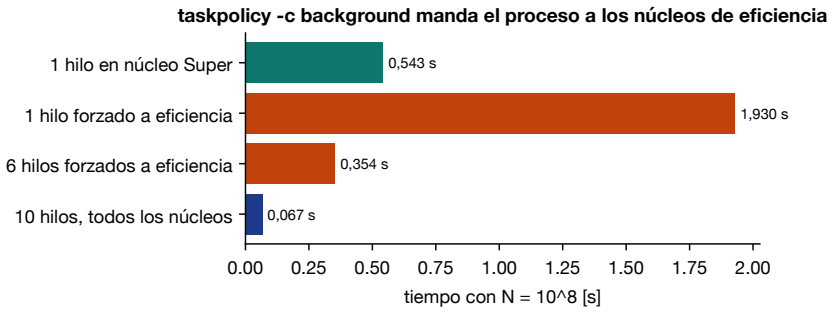


Figura 18. El mismo cálculo en un núcleo Super y forzado a los núcleos de eficiencia.

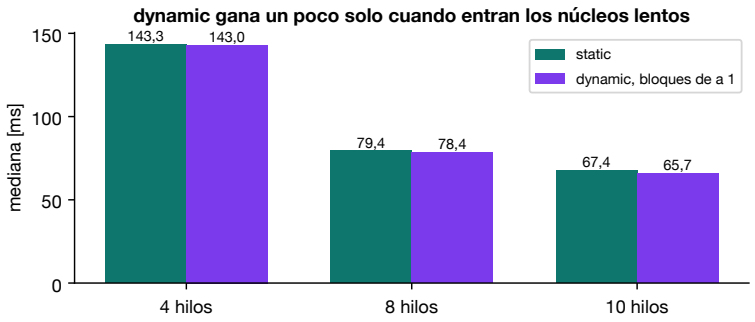


Figura 19. schedule(static) frente a schedule(dynamic). Con 4 hilos empatan y desde 8 dynamic gana por poco.

> **III.4.** Suponga que le ofrecen tiempo de cómputo en una máquina con 64 núcleos. Con su fracción f estimada, ¿qué speedup máximo podría obtener y en qué se diferenciaría de crecer el número de muestras N? Reflexione: aumentar p con N fijo, ¿mejora la precisión del resultado o solo el tiempo de cómputo? ¿Qué mejora cada una de las dos variables?

$$S(64) = 1 / (0{,}0172 + 0{,}983/64) = 1 / (0{,}0172 + 0{,}0154) = \mathbf{30{,}7} \qquad \text{techo} = 1 / 0{,}0172 \approx 58$$

Con 64 núcleos el programa iría unas **31 veces más rápido**, con eficiencia cercana al 48 %. Ese valor tiene un matiz. Mi f sale de un portátil con núcleos de dos velocidades, y parte de lo que Amdahl cuenta como secuencial es en realidad la diferencia entre núcleos. En un servidor de 64 núcleos iguales es probable que salga mejor.

p y N mejoran cosas distintas, como muestra la Figura 20.

- **Más hilos con el mismo N solo mejora el tiempo.** Con 1, 2, 4, 8 y 10 hilos el conteo fue idéntico, 78.541.825, y el tiempo bajó de 0,545 s a 0,067 s. El error no se movió de 0,0000803.
- **Más muestras mejora la precisión pero cuesta tiempo.** Con 10 hilos el error pasó de 0,00149 con 10^6 puntos a 0,000400 con 10^7 y a 0,0000803 con 10^8. Con 4×10^8 fue 0,0000040, que es buena suerte, porque lo esperado ronda 0,00008. El tiempo sí creció casi en proporción, de 0,067 s a 0,271 s.

Encontré un detalle adicional. Con $N = 10^6$, 10 hilos fueron apenas 2,2 veces más rápidos que uno, porque el cálculo dura 7 ms y los costos fijos pesan mucho más. Con tan poco trabajo, repartirlo casi no paga.

p decide qué tan rápido y N decide qué tan preciso. Lo sensato con 64 núcleos sería aumentar N al mismo tiempo, para gastar los núcleos en ganar precisión en el mismo tiempo de pared. Ahí aparece el límite de mi generador, porque la secuencia de 2^{32} valores alcanza para unos $7{,}2 \times 10^8$ puntos. Para ir más allá tocaría cambiar a un generador de período más largo.

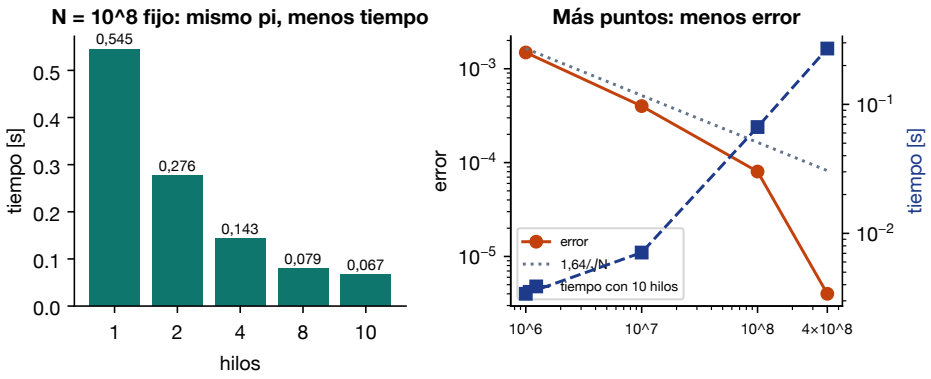


Figura 20. Izquierda: más hilos, mismo resultado y menos tiempo. Derecha: más puntos, menos error y más tiempo.

Parte IV. Informe sobre mi proceso de solución

> Redacte un informe breve, de una a dos páginas, en el que usted explique el trabajo que realizó para llegar a su solución. No se trata de repetir la teoría: se califica la claridad con la que usted da cuenta de su propio proceso. Su informe debe describir, como mínimo:
>
> 1. las decisiones de diseño que tomó y por qué las tomó: estructura del programa, cláusulas de OpenMP empleadas y manejo de las semillas de los generadores de cada hilo;
> 2. los problemas que aparecieron durante el desarrollo y cómo los diagnosticó y corrigió (por ejemplo: resultados distintos en cada ejecución, estimaciones sesgadas, condiciones de carrera) y qué evidencia le permitió identificar cada uno;
> 3. el método que usó para validar la correctitud de sus resultados, tanto la estimación de π como la reproducibilidad con distinto número de hilos;
> 4. la conexión explícita entre su informe y sus datos: cite valores concretos de su tabla (por ejemplo, por qué su mediana con $p = 4$ resultó como resultó) en lugar de descripciones genéricas que podrían aplicarse a cualquier entrega;
> 5. una reflexión final sobre lo que aprendió y sobre lo que haría de manera diferente en un próximo trabajo paralelo.

1. Decisiones de diseño. Quise tocar lo menos posible el programa del enunciado. El ciclo, la prueba $x^2 + y^2 \leq 1$ y las dos llamadas al generador por punto siguen iguales. La primera decisión la impuso mi equipo. Como el rand_r de macOS es otro algoritmo, escribí rand_r_glibc, una copia del de glibc, para que mis números sean los mismos que daría Linux. Después partí los 10^8 puntos en 400 bloques de 250.000, un número que se divide exacto entre 1, 2, 4, 8 y 10, y cada vuelta del `omp parallel for` procesa un bloque. Usé `reduction(+:dentro)` para que cada hilo cuente en su copia y se sume una sola vez, que es más barato que critical o atomic. Elegí `schedule(static)` por defecto porque todos los bloques cuestan lo mismo y porque así cada hilo recibe un tramo contiguo, separado al menos N/p puntos del siguiente. El número de hilos entra como argumento o por `OMP_NUM_THREADS`. Para las semillas, como el resultado tenía que ser igual con cualquier p, até los números al bloque y no al hilo. La función avanzar_lcg lleva cada bloque a su posición en la secuencia en unas 30 operaciones.

2. Problemas, diagnóstico y evidencia.

Problema	Cómo lo noté	Qué hice
rand_r distinto en macOS	El rand_r del sistema no seguía el algoritmo de glibc y con la misma semilla mis números no serían los de Linux	Escribí rand_r_glibc y la validé reproduciendo los 78.541.707 aciertos que una compañera obtuvo en Linux con la semilla 2224510
Semilla	El enunciado trae 20261018 y el profesor aclaró que cada uno usa su código	Usé 2211853 en todos los programas
Carrera sobre el estado que no se veía	La primera versión del experimento con estado compartido daba con 8 hilos un conteo fijo, 78.542.704, igual al de la misma semilla. El compilador había copiado el estado en un registro por hilo	Puse la llamada al generador en una función sin inline para que leyera y escribiera en memoria. Ahí sí cambió en cada corrida
Contador sin reduction	Con 1 hilo daba bien. Con 8, $\pi \approx 0{,}39$ y unos 9,82 millones de aciertos, la octava parte	reduction(+:dentro)
Núcleo frío	$S(2) = 2{,}07$, más que lineal. El secuencial siempre medía de primero tras una pausa	Corrida de calentamiento y orden rotativo. T(1) pasó de 0,5717 a 0,5451 s
Medición de costos en cero	Mi primer programa de costos marcaba 0 μs al crear los hilos, imposible	Lo reescribí para que cada hilo hiciera un trabajo visible y verifiqué que el reloj de omp_get_wtime resuelve 1 μs
gcc más lento que clang	Con un hilo, el mismo código tardaba 0,76 s con gcc 16 y 0,57 s con clang, según resultados/gcc_vs_clang.txt	Medí con clang y dejé gcc solo para comprobar el comando del enunciado

3. Validación. Con $N = 10^8$ obtuve $\pi = 3{,}141673$ con error de 0,0000803, menor que el típico esperado de 0,000164. Para la reproducibilidad no me bastó con que π se pareciera. Exigí que el conteo entero fuera idéntico, y fue **78.541.825** en el secuencial, con 1, 2, 4, 8 y 10 hilos, con static, con dynamic, con `OMP_NUM_THREADS` y en las 63 corridas de la Parte III. La suma de los 4 tramos de la Parte I da el mismo número. El programa verifica al arrancar que el salto del generador coincida con avanzar paso a paso. Compila sin advertencias con `-Wall -Wextra` en clang y en gcc 16, y esa salida vacía queda en `resultados/verificaciones.txt`. Como el M5 puede fusionar x·x + y·y en una sola instrucción, también compilé con `-ffp-contract=off` y el conteo no cambió.

4. Conexión con mis datos. Con $p = 4$ los siete tiempos quedaron entre 0,1428 y 0,1454 s, una dispersión muy pequeña, y la mediana fue 0,1433 s. Por eso **$S(4) = 0{,}5451 / 0{,}1433 = 3{,}80$** y no 4. Crear los hilos costó 33 μs, un 0,02 %, y dynamic dio 143,0 ms contra 143,3. Ni la creación de hilos ni el desbalance explican el 5 % que falta. Ese resto lo atribuyo a que macOS no fija los hilos a los núcleos y a la actividad del sistema. Con $p = 8$ la historia es otra. $S(8) = 6{,}87$ queda por debajo de Amdahl, que daba 7,14, porque cuatro de los ocho hilos caen en núcleos de eficiencia, que rinden cerca de 0,7 de un núcleo Super. El modelo $4 + 0{,}7 \cdot 4 = 6{,}8$ lo explica casi completo. La mediana también importó. La corrida de 106,6 ms con dynamic habría subido el promedio a 82,4 ms y cambiado la conclusión entre static y dynamic.

5. Reflexión. Lo primero que me llevo es que la reproducibilidad toca diseñarla desde el comienzo. Atar los números al bloque y no al hilo fue lo que la hizo posible para cualquier p. También aprendí que un error de concurrencia puede esconderse muy bien. Sin reduction el programa acierta con un hilo, y en el experimento de estado compartido el compilador llegó a esconder la carrera detrás de un resultado que parecía estable. Medir tampoco es tan directo como pensaba. Un núcleo frío me dio un speedup imposi-

ble y lo descubrí solo porque el número no tenía sentido. En un próximo trabajo revisaría primero qué tipo de núcleos tiene la máquina antes de interpretar el speedup, usaría un generador pensado para paralelo y sin el límite de 2^{32}, y mediría la frecuencia con permisos de administrador para confirmar lo que aquí solo pude inferir.

Anexo. Bitácora de uso de IA

Herramienta y fecha	Propósito	Qué aportó	Qué acepté o descarté y por qué
Claude Code, 27/10/2026	Partir de una referencia	Revisó el repositorio público de una compañera del curso, Nancy Saenz, y lo usó como base para la estructura de la entrega y la idea de repartir por bloques con salto del generador	Acepté esa idea y su conteo, 78.541.707 con la semilla 2224510, para validar mi generador. Reescribí el programa para macOS con rand_r_glibc, otro tamaño de bloque y p = 10, y todas las mediciones son nuevas, hechas en mi equipo con mi semilla
Claude Code, 27/10/2026	Parte I	Programas de apoyo para la curva de error, los 4 tramos y las figuras	Acepté apoyar cada respuesta con un experimento propio. La curva de error usa tramos disjuntos de la secuencia para que las 30 repeticiones no compartan números
Claude Code, 27/10/2026	Parte II	Programa paralelo, copia de rand_r de glibc y programas que cometen cada defecto	Acepté el diseño por bloques con 400 bloques. Corregí el experimento de carrera cuando el compilador la ocultaba
Claude Code, 27/10/2026	Parte III	Script de medición, medición de costos y de núcleos, y gráficas	Descarté la primera serie de tiempos por el núcleo frío y el primer programa de costos porque marcaba cero
Claude Code, 27/10/2026	Parte IV y redacción	Borrador del documento y de la presentación	Acepté la estructura. Revisé que cada cifra coincidiera con las salidas guardadas en el repositorio